

Basic commands: Once installed, users can begin utilising Podman with familiar commands.

To execute a container, the following command is employed:

```
podman run
```

For example, to run an Nginx container, use:

```
podman run -d -p 8080:80 nginx
```

To view currently running containers, the following command can be used:

```
podman ps
```

To halt a running container, the command is:

```
podman stop <container_id>
```

To delete a container, the command is:

```
podman rm <container_id>
```

Working with pods: Creating and managing pods in Podman is straightforward. To create a new pod, the following command is used:

```
podman pod create --name mypod
```

Containers can then be executed within this pod:

```
podman run -d --pod mypod nginx
podman run -d --pod mypod redis
```

Applications of Podman

Podman is utilised across various scenarios, including:

Development environments: Developers leverage Podman to create isolated environments for testing and development, eliminating the overhead associated with a daemon.

CI/CD pipelines: Podman can be seamlessly integrated into continuous integration and continuous deployment (CI/CD) pipelines, facilitating efficient

container management throughout the build and deployment processes.

Microservices architecture: With its support for pods, Podman is ideally suited for deploying microservices, where multiple containers must communicate within the same network namespace.

Kubernetes deployments: Podman's ability to generate Kubernetes manifests simplifies the transition from local development to production deployments in Kubernetes clusters.

Security-conscious environments: Organisations that prioritise security can benefit from Podman's rootless container support, allowing for safer execution of containers.

Use cases of Podman

Development and testing: Podman is an excellent choice for developers who need to create isolated environments for testing applications. Its rootless capabilities allow developers to run containers without requiring elevated privileges.

Microservices deployment: With its support for pods, Podman is well-suited for deploying microservices architectures. Developers can group related containers together, simplifying communication and resource sharing.

CI/CD pipelines: Podman can be integrated into continuous integration and continuous deployment (CI/CD) workflows, allowing for automated testing and deployment of containerised applications.

Cloud-native applications: Podman is compatible with Kubernetes, making it a suitable choice for developing and deploying cloud-native applications. Users can easily transition from local development to production environments.

Security-focused environments: Organisations that prioritise security can leverage Podman's rootless containers and daemonless architecture to minimise risks associated with container management.

Comparison with other container management tools

Podman vs Docker: While both Podman and Docker are popular container management tools, Podman's daemonless architecture and rootless capabilities provide enhanced security. Additionally, Podman's pod concept allows for better management of multi-container applications.

Podman vs Kubernetes: Podman is often used for local development and testing, while Kubernetes is a full-fledged orchestration platform for managing containerised applications in production. Podman can generate Kubernetes manifests, making it easier to transition from local development to Kubernetes deployments.

Podman vs Singularity: Singularity is another container management tool designed for high-performance computing (HPC) environments. While both tools support running containers without root privileges, Podman is more focused on general-purpose container management, whereas Singularity is tailored for scientific applications.

Podman represents a versatile and powerful tool for managing containers and pods, offering a range of features that align with contemporary development practices. Its daemon-less architecture, compatibility with Docker, and emphasis on security position Podman as a compelling choice for developers and system administrators. As containerisation continues to evolve, Podman is well-positioned to play a significant role in the container ecosystem, particularly in environments that prioritise security and simplicity. 

References

- <https://docs.podman.io/>
- <https://podman-desktop.io/>

By: Neetesh Mehrotra

The author is working in EY as a senior. His areas of interest are Java development and automation testing.